

Concurrency Control in DBMS

In a Database Management System (DBMS), Concurrency control is a mechanism in DBMS that allows simultaneous execution of transactions while maintaining ACID properties - Atomicity, Consistency, Isolation and Durability. It maintains the integrity, accuracy and reliability of data when multiple users or processes perform read/write operations concurrently. It helps manage issues like:

- Conflicting operations on shared data
- Inconsistent database states
- Lost or incorrect updates

Note: By implementing concurrency control techniques such as locking or timestamp ordering, DBMS ensures that transactions are executed safely and independently, even when they overlap in time.

Need of Concurrency Control

Concurrency control is essential to:

Scenario	Without Concurrency Control	With Concurrency Control
Initial Balance	\$1000	\$1000
Transaction A (User 1)	Withdraws \$200: Reads balance as \$1000, calculates new balance \$800.	Withdraws \$200: Reads balance as \$1000, calculates new balance \$800.
Transaction B (User 2)	Withdraws \$300: Reads balance as \$1000 (before A updates), calculates \$ 700	Waits for Transaction A to finish. Once A commits, reads updated balance \$800, calculates new balance \$500.
Final Updates	A writes \$800. Then, B reads overwrites with \$700 (lost update issue).	A writes \$800. Then, B reads updated balance and writes \$500 (consistent result).
Final Balance	\$700 (incorrect, inconsistent).	\$500 (correct, consistent).

Concurrency Control

- Prevent conflicts between simultaneous transactions.
- Maintain data consistency and accuracy in multi-user environments.
- Avoid problems such as dirty reads, lost updates and inconsistent reads.

Example:

- **Without concurrency control:** Two users update the same record simultaneously and one update overwrites the other.

- **With concurrency control:** The DBMS uses locks or timestamps to ensure updates occur sequentially and data remains correct.

Concurrency Problems in DBMS

When multiple transactions execute concurrently, several problems may occur:

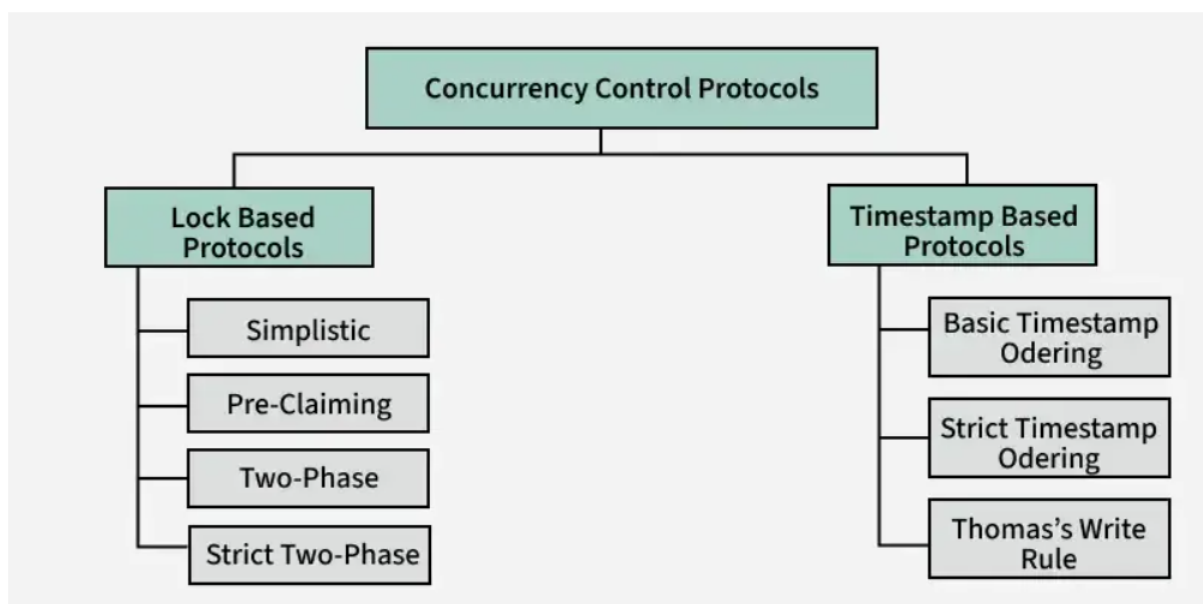
- **Dirty Read:** A transaction reads uncommitted data from another transaction that may later roll back.
- **Lost Update:** Two transactions update the same data and one update overwrites the other.
- **Inconsistent Read:** A transaction reads the same data multiple times and the data changes in between reads.

Concurrency Control Protocols

Concurrency control protocols define rules to ensure correct and consistent execution of transactions. Concurrency Control Protocols are the methods used to manage multiple transactions happening at the same time. They ensure that transactions are executed safely without interfering with each other, maintaining the accuracy and consistency of the database.

These protocols prevent issues like data conflicts, lost updates or inconsistent data by controlling how transactions access and modify data.

The main protocols are:



Concurrency Control Protocols

1. Lock-Based Concurrency Control:

- Uses locks to restrict access to data items during a transaction. Common types include shared locks (read) and exclusive locks (write).
- Ensures serializability and prevents conflicts.
- **Example:** Two-Phase Locking (2PL) guarantees that once a transaction releases a lock, it cannot obtain any new locks.

2. Timestamp-Based Concurrency Control:

- Each transaction is assigned a timestamp.
- The DBMS uses these timestamps to order transactions and prevent conflicts based on their start time.

Lock Based Concurrency Control Protocol in DBMS

A lock in DBMS controls concurrent access, allowing only one transaction to use a data item at a time. This ensures data integrity and prevents issues like lost updates or dirty reads during simultaneous transactions.

Lock Based Protocols in DBMS ensure that a transaction cannot read or write data until it gets the necessary lock. Here's how they work:

- These protocols prevent concurrency issues by allowing only one transaction to access a specific data item at a time.
- Locks help multiple transactions work together smoothly by managing access to the database items.
- Locking is a common method used to maintain the serializability of transactions.
- A transaction must acquire a read lock or write lock on a data item before performing any read or write operations on it.

Types of Lock

1. **Shared Lock (S):** Shared Lock is also known as Read-only lock. As the name suggests it can be shared between transactions because while holding this lock the transaction does not have the permission to update data on the data item. S-lock is requested using lock-S instruction.
2. **Exclusive Lock (X):** Data item can be both read as well as written. This is Exclusive and cannot be held simultaneously on the same data item. X-lock is requested using lock-X instruction.

Rules of Locking

The basic rules for Locking are given below:

Read Lock (or) Shared Lock(S)

- If a Transaction has a Read lock on a data item, it can read the item but not update it.
- If a transaction has a Read lock on the data item, other transaction can obtain Read Lock on the data item but no Write Locks.
- So, the Read Lock is also called a Shared Lock.

Write Lock (or) Exclusive Lock (X)

- If a transaction has a write Lock on a data item, it can both read and update the data item.
- If a transaction has a write Lock on the data item, then other transactions cannot obtain either a Read lock or write lock on the data item.
- So, the Write Lock is also known as Exclusive Lock.

Lock Compatibility Matrix

- A transaction can acquire a lock on a data item only if the requested lock is compatible with existing locks held by other transactions.
- **Shared Locks (S):** Multiple transactions can hold shared locks on the same data item simultaneously.
- **Exclusive Lock (X):** If a transaction holds an exclusive lock on a data item, no other transaction can hold any type of lock on that item.
- If a requested lock is not compatible, the requesting transaction must wait until all incompatible locks are released by other transactions.
- Once the incompatible locks are released, the requested lock is granted.

	S	X
S	✓	✗
X	✗	✗

Compatibility Matrix

Two Phase Locking Protocol

The Two-Phase Locking (2PL) Protocol is a key technique used in DBMS to manage how multiple concurrent transactions access and modify data. When many users or processes interact with a database, it's important to ensure that data remains consistent and error-free. Without proper management, issues like data conflicts or corruption can occur.

The Two-Phase Locking Protocol resolves this issue by defining clear rules for managing data locks. It divides a transaction into two phases:

1. **Growing Phase:** In this step, the transaction gathers all the locks it needs to access the required data. During this phase, it cannot release any locks.
2. **Shrinking Phase:** Once a transaction starts releasing locks, it cannot acquire any new ones. This ensures that no other transaction interferes with the ongoing process.

Types of Lock

- **Shared Lock (S):** Shared Lock is also called a read-only lock, allows multiple transactions to access the same data item for reading at the same time. However, transactions with this lock cannot make changes to the data. A shared lock is requested using the lock-S instruction.
- **Exclusive Lock (X):** An Exclusive Lock allows a transaction to both read and modify a data item. This lock is exclusive, meaning no other transaction can access the same data item while this lock is held. An exclusive lock is requested using the lock-X instruction.

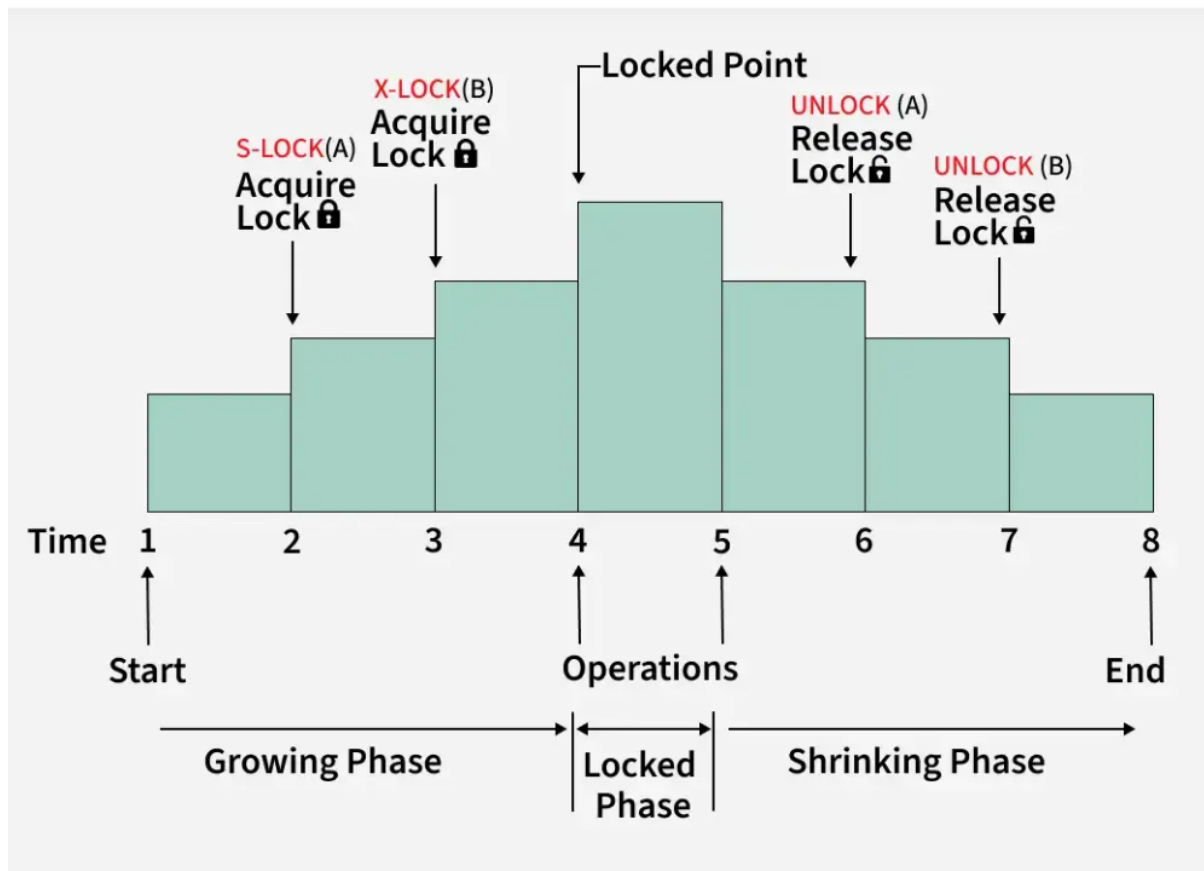
Lock Conversions

In the Two-Phase Locking Protocol, lock conversion means changing the type of lock on data while a transaction is happening. This process is carefully controlled to maintain consistency in the database.

1. **Upgrading a Lock:** This means changing a shared lock (S) to an exclusive lock (X). For example, if a transaction initially only needs to read data (S) but later decides it needs to update the same data, it can request an upgrade to an exclusive lock (X). However, this can only happen during the Growing Phase, where the transaction is still acquiring locks.
Example: A transaction reads a value (S lock) but then realizes it needs to modify the value. It upgrades to an X lock during the Growing Phase.
2. **Downgrading a Lock:** This means changing an exclusive lock (X) to a shared lock (S). For instance, if a transaction initially planned to modify data (X lock) but later decides it only needs to read it, it can downgrade the lock. However, this must happen during

the Shrinking Phase, where the transaction is releasing locks.

Example: A transaction modifies a value (X lock) but later only needs to read the value, so it downgrades to an S lock during the Shrinking Phase.



2PL-Locking

These rules ensure that the Two-Phase Locking Protocol maintains consistency and avoids conflicts between transactions. By limiting when upgrades and downgrades can occur, the system prevents situations where multiple transactions interfere with each other's operations.

Let's see a transaction implementing 2-PL.

	T1	T2
1	lock-S(A)	
2		lock-S(A)
3	lock-X(B)	
4		
5	Unlock(A)	
6		Lock-X(C)
7	Unlock(B)	
8		Unlock(A)
9		Unlock(C)
10		

This is a basic outline of a transaction that demonstrates how locking and unlocking work in the Two-Phase Locking Protocol (2PL).

Transaction T₁

- The growing Phase is from steps 1-3
- The shrinking Phase is from steps 5-7
- Lock Point at 3

Transaction T₂

- The growing Phase is from steps 2-6
- The shrinking Phase is from steps 8-9
- Lock Point at 6

Lock Point

- The lock point in a transaction is the moment when the transaction finishes acquiring all the locks it needs. After this point, no new locks can be added, and the transaction starts releasing locks. It's a key step in the Two-Phase Locking Protocol to ensure the rules of growing and shrinking phases are followed.

Example of 2PL

Imagine a library system where multiple users can borrow or return books. Each action (like borrowing or returning) is treated as a transaction. Here's how the Two-Phase Locking Protocol (2PL) works, including the lock point:

User A wants to:

1. Check the availability of Book X.
2. Borrow Book X if it's available.
3. Update the library's record.

Growing Phase (Locks are Acquired):

1. User A locks Book X with a shared lock (S) to check its availability.
2. After confirming the book is available, User A upgrades the lock to an exclusive lock (X) to borrow it.
3. User A locks the library's record to update the borrowing details.

Lock Point: Once User A has acquired all the necessary locks (on Book X and the library record), the transaction reaches the lock point. No more locks can be acquired after this.

Shrinking Phase (Locks are Released):

1. User A updates the record and releases the lock on the library's record.
2. User A finishes borrowing and releases the exclusive lock on Book X.

This process ensures that no other user can interfere with Book X or the library record during the transaction, maintaining data accuracy and

consistency. The lock point ensures that all locks are acquired before any are released, following the 2PL rules.

Drawbacks of 2PL

Two-phase locking (2PL) ensures that transactions are executed in the correct order by using two phases: acquiring and releasing locks.

However, it has some drawbacks:

- **Deadlocks:** Transactions can get stuck waiting for each other's locks, causing them to freeze indefinitely.
- **Cascading Rollbacks:** If one transaction fails, others that depend on it might also fail leading to inefficiency and potential data issues.
- **Lock Contention:** Too many transactions competing for the same locks can slow down the system, especially when many users are working at the same time.
- **Limited Concurrency:** The strict rules of 2PL can reduce how many transactions can run at once, resulting in slower performance and longer wait times.

Graph Based Concurrency Control Protocol in DBMS

In a Database Management System (DBMS), multiple transactions often execute concurrently, which can lead to conflicts when they access the same data items. The Graph-Based Concurrency Control Protocol manages these conflicts using a directed graph structure to ensure conflict serializability and consistency.

- Transactions -> Nodes
- Conflicts (dependencies) -> Edges

Before granting a lock, the system checks the graph:

- If adding a new edge creates a cycle, a deadlock is detected.
- To resolve it, one of the transactions is rolled back to break the cycle.

Note: *Thus, the protocol ensures that the serialization graph remains acyclic, maintaining database consistency.*

Properties

- **Deadlock-Free:** Since cycles are not allowed, deadlocks cannot occur.
- **No guarantee of recoverability:** Transactions can still cause cascading rollbacks.
- **More powerful** than simple lock-based protocols because it can handle complex dependencies.
- **Computationally expensive** for large databases due to cycle detection and graph maintenance.

Partial Ordering Requirement

Before applying the protocol, a partial ordering is defined on the set of data items. Let the set of database items be $D = \{d_1, d_2, d_3, \dots, d_n\}$. If $d_i \rightarrow d_j$, then any transaction accessing both must access d_i before d_j .

Note: *This ordering forms a Directed Acyclic Graph (DAG) - called the Database Graph.*

Tree Based Protocol

The Tree Protocol is a graph-based concurrency control method that ensures conflict serializability and deadlock freedom. Unlike Two-Phase Locking (2PL), it follows a hierarchical structure for locking data items.

Rules of Tree Protocol:

1. **Only exclusive (X) locks are allowed:** No shared (S) locks are used.
2. **The first lock can be on any data item:** After that, a transaction can lock a data item only if it has locked its parent first.
3. **Data items can be unlocked at any time:** There is no strict two-phase rule.
4. **No relocking:** Once a transaction unlocks a data item, it cannot lock it again.

Note: The Tree Protocol offers a balance between concurrency and deadlock prevention, making it useful in certain database applications.

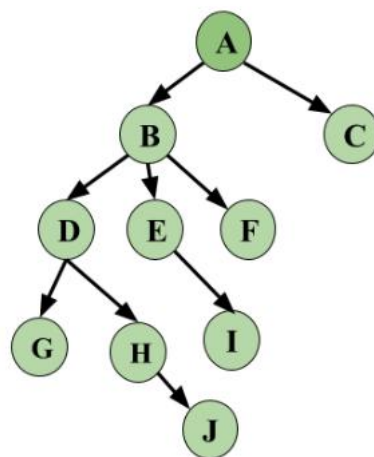


Image - Database Graph

Let's look at an example based on the above Database Graph. We have three Transactions in this schedule and this is a skeleton example, i.e, we will only see how Locking and Unlocking work, let's keep this simple and not make this complex by adding operations on data.

Advantages of Tree Protocol

- **Deadlock-free** – No cycles form in the lock order, so no rollbacks are needed.
- **Increased concurrency** – Unlocking can happen earlier than in Two-Phase Locking, reducing wait times.
- **Allows different schedules** – Some schedules that are not possible under 2PL can be executed under the tree protocol.

Drawbacks of Tree Protocol

- **No guarantee of recoverability** – Can lead to cascading rollbacks if not handled properly.
- **Extra locks required** – Transactions may need to lock unnecessary data items, increasing waiting time and locking overhead.
- **Some schedules allowed in 2PL are not possible in Tree Protocol** – While it provides flexibility, it also has limitations.